

Project 1: The Mystery Binary

EECE.4821/5821 Computer Architecture

Released: **09/23/2026** Due: **10/07/2026, 11:59 pm** (two weeks)

Individual assignment 100 points (4821) / 130 points (5821)

The story

A prototype RISC-V controller was recovered from the lab, but its firmware source code has been lost. All that is left is a memory dump: a list of 32-bit machine words from the text segment and a few words from the data segment. Your job is to work out what the firmware does: decode it by hand, rebuild source code that assembles to the same bits, run it in a simulator, measure how fast it is, and then make it faster.

Every student gets a different binary. The algorithm is the same for everyone, but register assignments, data addresses and output strings differ between variants. Your `mystery.txt` is yours alone. You may discuss *techniques* with classmates, but all decoding, code and numbers must be your own work.

Learning objectives

1. Decode RV32I machine code by hand: identify the format, extract the fields, and sign-extend and reassemble the immediates.
2. Write RISC-V assembly that reproduces a given binary exactly, then use a simulator (OARS) to single-step, set breakpoints and inspect registers and memory.
3. Count dynamic instructions analytically and check the counts in the simulator.
4. Use the CPU performance equation $\text{CPU time} = \text{IC} \times \text{CPI} \times T_{\text{clk}}$ to evaluate and justify an optimization.

1 Getting started with oars

OARS (Oxide Assembler and Runtime Simulator) is a modern, single-binary successor to RARS. Download the release for your platform from <https://github.com/hutch-hutchins/OARS/releases/latest> and unzip it. There is nothing to install. (macOS: right-click → Open the first time, or run `xattr -cr oars`.) The `examples/32bit` folder that ships with it is worth a few minutes of your time.

Editor → Assemble

Assembling switches to the **Text Segment** tab, which lists the address, *machine code* and source of every instruction. This is how you check your reconstruction bit for bit.

Step / Backstep / breakpoints

Click a row in the Text Segment view to set a breakpoint. Registers that changed are highlighted in green.

Registers → CSR tab

shows `instret`, the number of instructions retired so far. Use it to measure dynamic instruction counts.

Memory / Data Segment tabs

show the `.data` contents, including your array.

Console

Your program reads n here (syscall 5) and prints its result.

Region	Base address	a7	Syscall used in this project
<code>.text</code>	0x00400000	1	print integer in <code>a0</code>
<code>.data</code>	0x10010000	4	print NUL-terminated string at <code>a0</code>
heap	0x10040000	5	read integer into <code>a0</code>
stack top	0x7FFFEFFC	10	exit (11 = print character in <code>a0</code>)

oars details that matter for this project

- In the GUI, execution starts at the label `main`. Put `main:` on the first instruction of `.text`.
- OARS does **not** count the final `exit` `ecall` in `instret` (RARS behaves the same way). Every instruction count in this project follows that rule.
- OARS v0.8 does not align `.word` automatically, and labels that come *after* an `.align` directive get the wrong address. Lay out `.data` with `.ascii`, `.word` and `.space` only, and add padding yourself.
- Pseudo-instructions such as `li`, `la` and `j` can expand differently from what the binary contains. In `reconstructed.s`, write the *base* instructions exactly as you decoded them.

2 Your starter pack

<code>mystery.txt</code>	The dump: one <code>address:</code> word line per instruction, plus the initial <code>.data</code> words (little-endian). The header line holds your variant ID .
<code>decode.csv</code>	Template for Part A (addresses and words already filled in).
<code>reconstructed.s</code> , <code>optimized.s</code>	Skeletons for Parts B and D.
<code>answers.txt</code>	Template for your numeric answers (Parts C and D). Keep the key names.
<code>sweep.csv</code> , <code>optimized_rv32i.s</code>	(5821 only) templates for G1 and G3; <code>answers.txt</code> also has the G1/G2 keys.

The program reads one integer n with $0 \leq n \leq 1000$ and prints one line of output.

3 Part A: Decode the binary

25 pts

Fill in the last two columns of `decode.csv` for *every* word in the text segment.

- **format:** one of R I S B U J. Loads, `jalr`, shifts-by-immediate and `ecall` are I-type.
- **assembly:** the *base* instruction, operands in the usual order. Registers may be written `x10` or `a0`. Immediates may be decimal or hex. Branch/`jal` targets may be the absolute target address (0x0040003c) *or* the signed byte offset (+36). `lui`/`auipc` take the 20-bit field (0x10011). Loads and stores use `offset(base)`.

```
address,hex,format,assembly
0x00400000,0x00500893,I,addi a7, zero, 5
0x00400018,0x034ac263,B,blt s5, s4, 0x0040003c
```

Worked example: a B-type word (not from anyone's binary)

0xFE0A08E3 at address 0x00400050:

$$\begin{array}{cccccc} \underbrace{1111111}_{} & \underbrace{00000}_{} & \underbrace{10100}_{} & \underbrace{000}_{} & \underbrace{10001}_{} & \underbrace{1100011}_{} \\ \text{imm}[12|10:5] & \text{rs2}=x0 & \text{rs1}=x20 & \text{beq} & \text{imm}[4:1|11] & \text{BRANCH} \end{array}$$

Reassembling: $\text{imm}[12]=1$, $\text{imm}[11]=1$, $\text{imm}[10:5]=111111$, $\text{imm}[4:1]=1000$, $\text{imm}[0]=0$, so $\text{imm} = 1\,1111\,1111\,0000_2 = -16$. The instruction is **beq s4, zero, -16**, and its target is $0x00400050 - 16 = 0x00400040$.

In your report, show the full field-by-field work for at least **one instruction of each format** (R, I, S, B, U, J), in the style of the example above.

4 Part B: Reconstruct the source

20 pts

Write **reconstructed.s** so that OARS assembles it to **exactly** the words in **mystery.txt**, in the same order and at the same addresses, and so that it behaves identically for every $0 \leq n \leq 1000$.

- Use labels for branch and jump targets. Pick meaningful names (**loop**, **done**, ...) and comment every basic block with what it does.
- Rebuild **.data**: decode the initial words back into the characters they represent (mind the byte order). If the code touches memory past the initialised words, reserve that space with **.space** at the correct address.
- Check your work: in the Text Segment tab, compare the *machine code* column against **mystery.txt**, then run the program for several n .

Scoring: 12 pts for the fraction of machine words that match exactly, 8 pts for the fraction of hidden test inputs that produce identical output.

5 Part C: Analyze the original program

25 pts

Instruction classes and the CPI of each (a multi-cycle implementation, 500 MHz clock):

Class	Instructions	CPI
ALU	R-type integer ops (add , sub , ...), I-type arithmetic/logic/shift, lui , auipc	4
MulDiv	mul* , div* , rem* (RV32M)	30
Load	lb lh lw lbu lhu	5
Store	sb sh sw	4
Branch	beq bne blt bge bltu bgeu	3
Jump	jal jalr	3
System	ecall (except the final exit), csrr*	10

- C1. What does it compute?** Give a C-like equivalent of the program and a one-line mathematical description of the output as a function of n . (*hand-graded*)
- C2. Measure.** Use `instret` to find the dynamic instruction count for $n = 10$ (`ic_n10`). Include a screenshot.
- C3. Derive.** Split the program into basic blocks and write down how many times each block runs as a function of n . From that, derive the closed form $IC(n) = an^2 + bn + c$ (`ic_a`, `ic_b`, `ic_c`), and check it against OARS for at least three values of n . (*derivation hand-graded*)
- C4. Instruction mix.** Using your block counts, give the number of instructions in each class for $n = 10$ (`mix_*`). The seven numbers must add up to `ic_n10`.
- C5. Performance.** For $n = 10$ compute total cycles, average CPI, and execution time in μs (`cycles_n10`, `cpi_n10`, `time_n10_us`). Also compute `orig_cycles_n100`, the cycles for $n = 100$.
- C6. Think.** (*hand-graded*) Which class contributes the most *cycles*? What fraction of the $n = 100$ run is spent inside the subroutine? Using Amdahl's law, what is the largest speedup you could get by making only that subroutine infinitely fast?

6 Part D: Optimize

20 pts

Write `optimized.s`, a program that produces *byte-for-byte* the same output as the original for every $0 \leq n \leq 1000$, but in as few **cycles** as possible under the CPI table above.

Rules. RV32I and RV32M only (no floating point). Read the input with exactly one syscall 5. No precomputed answer tables: at most 64 non-zero words of initialised `.data`. The program must compute the result.

Scoring. 8 pts for correctness on hidden inputs. The 12 performance points are awarded only if every correctness test passes, based on the speedup in total cycles summed over $n \in \{1, 10, 100, 1000\}$:

Speedup vs. original	$\geq 2\times$	$\geq 100\times$	$\geq 300\times$	$\geq 2000\times$
Points	3	6	9	12

In `answers.txt` report `opt_ic_n100`, `opt_cycles_n100` and `speedup_n100`. In the report, explain each optimization and why it helps (IC, CPI, or both?). Discuss whether RV32M is worth it: a `mul` costs 30 cycles, and a smaller instruction count does not automatically mean a faster program. For which values of n is your version *not* much faster than the original, and why?

7 Graduate component (EECE.5821)

+30 pts

All three parts build on work you have already done in Parts C and D. Most of the points are autograded, so use the file formats exactly as given.

G1: scaling study and plot (10 pts: 6 autograded, 4 hand-graded)

How does the benefit of your optimization depend on the input size?

- (a) Fill in the `sweep.csv` template from your starter pack. It has one row for each n in $\{0, 1, 2, 5, 10, 20, 50, 100, 200, 500, 1000\}$:

```
n,orig_ic,orig_cycles,opt_ic,opt_cycles,speedup
10,441,1616,...
```

`orig_*` is the original program, `opt_*` is your `optimized.s`, cycles use the machine-A CPI table from Part C, and `speedup` = `orig_cycles/opt_cycles`. Instruction counts must match OARS exactly. Get the cycles from instruction-mix formulas for each program; you already have one for the original from C3/C4.

- (b) In `answers.txt`, set `g1_crossover_n` to the smallest n in the table at which your optimized program is at least $10\times$ faster.
- (c) (*hand-graded*) Plot, on log-log axes, (i) cycles vs. n for both programs and (ii) speedup vs. n , with a line at $10\times$. Use any tool you like and include the figure in your report. Label the growth rate of each curve (e.g. $O(n^2)$) and explain, using the performance equation, why the speedup is small for small n . Is there any n where your “optimized” program is actually slower?

G2: which machine is faster? (10 pts: 8 autograded, 2 hand-graded)

Machine A is the implementation from Part C (CPI table above, 500 MHz). Machine B is a different implementation of the same ISA, with a lower CPI for most classes, expensive system calls, and a slower clock:

	ALU	MulDiv	Load	Store	Branch	Jump	System	Clock
Machine A	4	30	5	4	3	3	10	500 MHz
Machine B	1	12	2	1	2	2	20	300 MHz

For $n = 100$, report in `answers.txt`:

- original program: `a_orig_time_n100_us`, `b_orig_cycles_n100`, `b_orig_time_n100_us`, and `b_vs_a_orig_n100` = (time on A)/(time on B);
- your `optimized.s`: `b_opt_cycles_n100`, `b_opt_time_n100_us`, `b_vs_a_opt_n100`;
- `b_clock_match_mhz`: the clock rate machine B would need to run the *original* program in exactly the same time as machine A.

(*hand-graded*) Is one machine faster for both programs? Explain the answer with the instruction mix of each program. What does this say about quoting one “speedup” for a machine without naming the workload?

G3: optimize without the M extension (10 pts, autograded)

Write `optimized_rv32i.s`: same output as the original for every $0 \leq n \leq 1000$, using **RV32I only**. Any `mul`, `div` or `rem` executed scores 0 for performance. The other Part D rules also apply. You get 4 pts for correctness on the hidden inputs. Performance points (machine A cycles, same four inputs as Part D) are awarded only if every test passes:

Speedup vs. original	$\geq 100\times$	$\geq 300\times$	$\geq 1000\times$
Points	2	4	6

Hint: the top tier is reachable. Think about how many iterations a shift-and-add multiply really needs for $n \leq 1000$, and how to divide by a constant without a divide instruction. In the report, compare this version's IC, CPI and cycles with your `optimized.s`.

8 Deliverables and submission

Submit one `<studentLastname>_P1.zip` (for example: `boewe_P1.zip`) containing these files at the top level:

<code>decode.csv</code>	Part A (autograded)
<code>reconstructed.s</code>	Part B (autograded)
<code>answers.txt</code>	Parts C and D numbers, with your variant filled in (autograded)
<code>optimized.s</code>	Part D (autograded)
<code>report.pdf</code>	Written report: ≤ 6 pages (4821) or ≤ 10 pages (5821). Sections: decoding work (one instruction of each format), reconstruction notes, C1–C6 with block-count table and OARS screenshots, optimization discussion, and the G1–G3 write-ups with the G1 plot (5821).
<code>sweep.csv</code>	G1 table (5821 only, autograded)
<code>optimized_rv32i.s</code>	G3 (5821 only, autograded)

About the autograder

Most of the points are computed by a script that runs your files in OARS. File names, the CSV column order, and the key names in `answers.txt` must be *exactly* as given. Numbers only in `answers.txt`: no units and no formulas. Integer quantities must be exact. CPI and time are accepted within 1 %. If an early number is wrong, a later number computed correctly *from* it still earns partial credit, so show your work in the report.

Point summary

Component	4821	5821	Graded by
A. Decode	25	25	autograder
B. Reconstruct (encoding / behaviour)	12 + 8	12 + 8	autograder
C. Analysis (numbers / write-up)	17 + 8	17 + 8	autograder / instructor
D. Optimize (correct / speed)	8 + 12	8 + 12	autograder
Report quality	10	10	instructor
G1 scaling study (table / plot)	–	6 + 4	autograder / instructor
G2 machine comparison (numbers / discussion)	–	8 + 2	autograder / instructor
G3 RV32I-only optimization	–	10	autograder
Total	100	130	

Graduate scores are reported as a percentage of 130.